Helm for beginners

Package, configure, upgrade and roll back Kubernetes applications with Helm: charts, values, templates, releases, dependencies and debugging.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What Helm is and why teams use it (11 min)
2. Installing charts and overriding values (12 min)
3. Anatomy of a chart (12 min)
4. Templates: the Go template language in Helm (13 min)
5. Upgrade, rollback and release history (12 min)
6. Values per environment and keeping secrets out (13 min)
7. Dependencies, subcharts and registries (13 min)
8. Debugging Helm releases, hooks and tests (14 min)
9. Testing and linting Helm charts (12 min)
10. Library charts and reuse across services (12 min)
11. Publishing and signing Helm charts (12 min)
12. Helm security and supply chain (12 min)
Cheat sheet

Lesson 1. What Helm is and why teams use it

11 minutes. Understand charts, releases and repositories, and see how Helm turns a pile of Kubernetes YAML into one installable, upgradable package.

A real application on Kubernetes is not one YAML file. It is a Deployment, a Service, an Ingress, a ConfigMap, a ServiceAccount, maybe a HorizontalPodAutoscaler, and the values differ between development and production. Copying and editing those files for each environment leads to drift and mistakes. Helm is the package manager for Kubernetes: it bundles the files as a chart, fills in the differences from values, and tracks every install as a release you can upgrade or roll back.

What you will learn

- The problem Helm solves
- Charts, releases, repositories and values
- Install your first chart and look at what it created
- What Helm does not do

The problem

Plain manifests have three weaknesses. They repeat themselves (the same label in ten places), they cannot easily vary by environment (replica count, image tag, hostname), and they have no concept of "this set of objects is one application at version 1.4". Tools such as Kustomize patch plain YAML; Helm adds **templating** and **packaging**. With Helm, one chart describes the application, and a small values file describes how this installation differs. The same chart can be installed ten times in ten namespaces.

Four words to know

- A **chart** is a package: a folder (or a `.tgz` file) with templates and default values.
- A **release** is one installation of a chart in a cluster, with a name, such as `shop-web`. Install the same chart twice and you get two releases.
- A **repository** is a place that serves charts, either an HTTP index or an OCI registry.
- **Values** are the settings that fill the templates. Defaults come from the chart; you override them.

Helm 3 has no server component. The `helm` command talks to the Kubernetes API with your kubeconfig and stores the release history as Secrets in the release namespace.

Install something

Use a made-up repository name; the commands are the same for any repository.

```
helm repo add acme https://charts.example.com
helm repo update
helm search repo acme/web
helm install shop-web acme/web --namespace shop --create-namespace
helm list --namespace shop
helm status shop-web --namespace shop
kubectl get all --namespace shop
```

`helm install NAME CHART` renders the templates, sends the resulting manifests to the cluster and records revision 1. `helm list` shows releases, `helm status` shows the state and the notes the chart prints, and `kubectl` shows the real objects. Everything Helm created carries labels such as `app.kubernetes.io/managed-by: Helm` and an annotation with the release name.

MARGA by EknathaLabs
marga.eknathalabs.com

What Helm does not do

Helm does not make bad YAML good, does not manage secrets by itself, and does not watch the cluster afterwards: it applies changes when you run a command. If someone edits an object with `kubectl edit`, Helm will not notice until the next upgrade, which may overwrite the edit. Many teams run Helm from a pipeline or a controller that applies the chart on every change from version control, so the cluster always matches what was reviewed. Charts from the internet are code that runs in your cluster: read them, pin versions and prefer sources you trust.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How What Helm is and why teams use it fits together

Part	What it does	What goes wrong
The problem	Plain manifests have three weaknesses. They repeat themselves (the same label in ten places), they cannot easily vary by environment (replica count, image tag...	Editing live objects with kubectl and expecting Helm to know.
Four words to know	A chart is a package: a folder (or a .tgz file) with templates and default values.	Installing charts from unknown sources without reading them.
Install something	Use a made-up repository name; the commands are the same for any repository.	Copying manifests per environment.
What Helm does not do	Helm does not make bad YAML good, does not manage secrets by itself, and does not watch the cluster afterwards: it applies changes when you run a command.	Using latest as a chart or image version.

Flow: The problem to Four words to know / Four words to know to Install something / Install something to What Helm does not do

Incident: Three copies of the same YAML

Incident: *Staging and production drifted apart because each had its own set of copied manifests.*

```
$ diff -r k8s/staging k8s/prod | wc -l
143
$ grep -rn "replicas:" k8s/prod/deployment.yaml k8s/staging/deployment.yaml
k8s/prod/deployment.yaml:8:  replicas: 6
k8s/staging/deployment.yaml:8:  replicas: 2
```

Over months, people fixed one environment and forgot the other, so staging no longer tested what production ran. The team moved to a single chart with values-staging.yaml and values-prod.yaml. The diff between environments shrank to a short values file, and a fix to a template reaches both at once.

Common mistakes

- Copying manifests per environment
- Installing charts from unknown sources without reading them
- Editing live objects with kubectl and expecting Helm to know
- Using latest as a chart or image version

Check yourself

What is a Helm release?

Answer: One installation of a chart in a cluster, with a name and history

Where does Helm 3 store release history?

Answer: As Secrets in the release namespace

MARGA by EknathaLabs
marga.eknathalabs.com

What does a chart contain?

Answer: Templates, default values and metadata

Why use values files?

Answer: To vary the same chart per environment without copying templates

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Installing charts and overriding values

12 minutes. Find charts, read their default values, override them safely with files and --set, and preview what will be installed.

Most of your first week with Helm is spent answering one question: how do I change this setting? Every chart exposes its options as values. Knowing how to read them, override them and preview the result before it touches the cluster is the core skill, and it also protects you from the most common mistake: installing with defaults that are not meant for production.

What you will learn

- Search repositories and read a chart before installing
- Inspect default values with helm show values
- Override values with -f files and --set, and know the precedence
- Preview with helm template and --dry-run

Look before you install

```
helm repo add acme https://charts.example.com
helm repo update
helm search repo acme --versions | head
helm show chart acme/web --version 2.4.1
helm show readme acme/web --version 2.4.1
helm show values acme/web --version 2.4.1 > defaults.yaml
```

Always pin `--version`, so a repository update does not change what you install. Read the readme and the defaults: look for the image tag, replica count, resource requests, service type, ingress settings and any "persistence" or "auth" section. A chart that enables a persistent volume or a public LoadBalancer by default may not be what you want.

Override with a values file

Write only the keys you change, in a file under version control:

```
# values-prod.yaml
replicaCount: 4
image:
  repository: registry.example.com/shop/web
  tag: "1.8.2"
ingress:
  enabled: true
  host: shop.example.com
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    memory: 512Mi
```

```
helm install shop-web acme/web --version 2.4.1 -n shop -f values-prod.yaml
```

Maps are merged key by key, but **lists are replaced whole**, so overriding one item of a list means supplying the entire list. Quote version-like strings (`"1.8.2"`, `"true"`) so YAML does not turn them into numbers or booleans.

--set and precedence

For a one-off change use `--set`:

```
helm upgrade shop-web acme/web -n shop -f values-prod.yaml --set image.tag=1.8.3
helm upgrade shop-web acme/web -n shop --set-string podAnnotations.build=2026
helm upgrade shop-web acme/web -n shop --set "ingress.hosts[0]=shop.example.com"
```

MARGA by EknathaLabs
marga.eknathalabs.com

From lowest to highest priority: the chart's `values.yaml`, parent chart values, files given with `-f` (later files win), then `--set`. Pipelines usually use files for everything stable and `--set image.tag=...` for the one value that changes on every release. Avoid long `--set` lists; they are hard to review and reproduce.

Preview and verify

```
helm template shop-web acme/web --version 2.4.1 -f values-prod.yaml > rendered.yaml
helm install shop-web acme/web -n shop -f values-prod.yaml --dry-run --debug
helm get values shop-web -n shop -a        # all values in use
helm get manifest shop-web -n shop         # the YAML that was applied
```

`helm template` renders locally without a cluster, so you can lint it, run it through policy checks and review the diff in a pull request. After installing, `helm get values -a` and `helm get manifest` show what is really deployed. Review the rendered output for the things that matter: image tags, resource limits, security context and exposed services.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Installing charts and overriding values fits together

Part	What it does	What goes wrong
Look before you install	Always pin --version, so a repository update does not change what you install.	Not pinning chart versions.
Override with a values file	Write only the keys you change, in a file under version control.	Upgrading with only --set and losing earlier overrides.
--set and precedence	For a one-off change use --set.	Installing with unread defaults.
Preview and verify	helm template renders locally without a cluster, so you can lint it, run it through policy checks and review the diff in a pull request.	Using floating tags such as latest in values.

Flow: Look before you install to Override with a values file / Override with a values file to --set and precedence / --set and precedence to Preview and verify

Incident: The upgrade that reset the settings

Incident: *After a routine image bump, the production site lost its custom hostname and resource limits.*

```
$ helm upgrade shop-web acme/web -n shop --set image.tag=1.8.3
Release "shop-web" has been upgraded. Happy Helming!
$ helm get values shop-web -n shop
USER-SUPPLIED VALUES:
image:
  tag: 1.8.3
```

The engineer ran an upgrade with only --set and no values file, so Helm used the chart defaults for everything else: the earlier overrides were not carried over (--reuse-values would have kept them, with its own risks). The fix was a rule that every upgrade passes the same -f files plus the one changed value, stored in the repository, and a pipeline step that runs helm diff before applying.

Common mistakes

- Installing with unread defaults
- Not pinning chart versions
- Upgrading with only --set and losing earlier overrides
- Using floating tags such as latest in values

Check yourself

Which has the highest priority?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: --set on the command line

What happens to a list value you override?

Answer: The whole list is replaced

How do you render templates without a cluster?

Answer: helm template

How do you see the values really in use?

Answer: helm get values RELEASE -a

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Anatomy of a chart

12 minutes. Create a chart with helm create, understand each file, and know what to keep and what to delete.

To use Helm well you will soon need to read a chart or write a small one for your own application. A chart is just a directory with a fixed layout, so it is easier than it looks. Understanding what each file is for lets you read third-party charts quickly and write your own without cargo-culting the generated example.

What you will learn

- The files of a chart and their purpose
- Chart.yaml: version and appVersion
- values.yaml as the chart interface
- _helpers.tpl, NOTES.txt and helm lint

Generate and look

```
helm create shop-web
tree shop-web

shop-web/
  Chart.yaml          name, version, appVersion, description, dependencies
  values.yaml         default values: the interface of the chart
  charts/             packaged subcharts (dependencies)
  templates/
    deployment.yaml
    service.yaml
    ingress.yaml
    hpa.yaml
    serviceaccount.yaml
    _helpers.tpl      named template snippets, not rendered on their own
    NOTES.txt         message printed after install
    tests/test-connection.yaml
  .helmignore         files to leave out of the package
```

The generated chart is a good starting point but includes things you may not need. Delete what you do not use (the hpa, the test, the ingress) and keep the chart small.

Chart.yaml

```
apiVersion: v2
name: shop-web
description: The shop web frontend
type: application
version: 0.3.1        # version of the chart (packaging)
appVersion: "1.8.2"   # version of the application inside
```

`version` follows semantic versioning and must increase whenever the chart changes, because repositories and OCI registries treat versions as immutable. `appVersion` is informational: the version of the app (often the default image tag). Do not confuse them: you may release chart 0.3.2 with the same app, or app 1.9.0 with chart 0.4.0.

values.yaml is the interface

The values file documents every setting a user may change, with safe defaults and comments:

```
replicaCount: 2
image:
  repository: registry.example.com/shop/web
  tag: ""            # empty means use appVersion
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
resources: {}
```

MARGA by EknathaLabs marga.eknathalabs.com

Treat it like an API: keep names stable, group related keys, and avoid exposing every Kubernetes field. Fewer, well named values make a chart easier to use. Add `values.schema.json` to validate input, so a typo such as `replicaCont` fails with a message instead of being ignored.

Helpers, notes and linting

`_helpers.tpl` defines reusable snippets with `define`, such as the standard name and labels, used with `include`. `NOTES.txt` is a template printed after install; use it for how to reach the app. Check your work often:

```
helm lint shop-web
helm template shop-web ./shop-web | kubectl apply --dry-run=client -f -
helm package shop-web          # creates shop-web-0.3.1.tgz
```

`helm lint` finds missing fields and bad syntax; rendering and applying with a dry run catches Kubernetes schema errors. Run both in CI for every chart change.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Anatomy of a chart fits together

Part	What it does	What goes wrong
Generate and look	The generated chart is a good starting point but includes things you may not need.	Leaving unused generated templates in the chart.
Chart.yaml	version follows semantic versioning and must increase whenever the chart changes, because repositories and OCI registries treat versions as immutable.	Reusing the same chart version for changed content.
values.yaml is the interface	The values file documents every setting a user may change, with safe defaults and comments.	Exposing every Kubernetes field as a value.
Helpers, notes and linting	_helpers.tpl defines reusable snippets with define, such as the standard name and labels, used with include.	Skipping helm lint in CI.

Flow: Generate and look to Chart.yaml / Chart.yaml to values.yaml is the interface / values.yaml is the interface to Helpers, notes and linting

Incident: Same version, different chart

Incident: *Two clusters installed chart 1.2.0 a month apart and behaved differently.*

```
$ helm get manifest shop-web -n shop | grep -c "readinessProbe"
0
$ helm pull acme/web --version 1.2.0 --untar && grep -c readinessProbe web/templates/deployment.ya \
    ml
1
```

Someone had republished a different package under the same chart version, so the clusters held different content. The repository now refuses to overwrite an existing version, the pipeline bumps the chart version automatically from the change type, and packages are signed or stored in an OCI registry with immutable tags.

Common mistakes

- Reusing the same chart version for changed content
- Exposing every Kubernetes field as a value
- Leaving unused generated templates in the chart
- Skipping helm lint in CI

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does appVersion describe?

Answer: The version of the application packaged in the chart

What is _helpers.tpl for?

Answer: Reusable named template snippets

What does helm package do?

Answer: Creates a .tgz archive of the chart

Why add values.schema.json?

Answer: To validate values and fail on mistakes such as typos

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Templates: the Go template language in Helm

13 minutes. Write templates with values, conditionals, loops, helpers and the common functions, and avoid indentation and quoting traps.

Helm templates are Kubernetes YAML with small pieces of logic inside double curly braces. The logic is Go templating plus a library of helper functions. You do not need all of it. A dozen patterns cover nearly every chart, and the main skill is keeping the output valid YAML, because template whitespace and indentation are where most chart bugs hide.

What you will learn

- Insert values and built-in objects
- Use if, with and range
- Define and include helpers, and use toYaml with nindent
- Control whitespace and fail early with required

Values and objects

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-web
  labels:
    app.kubernetes.io/name: {{ .Chart.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: web
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion  \
    }}"
```

The top-level objects are `.Values` (your values), `.Release` (name, namespace, revision, whether it is an upgrade), `.Chart` (Chart.yaml fields), and `.Capabilities` (cluster version and API support). The pipe `|` passes a value into a function: `default`, `quote`, `upper`, `trunc`, `sha256sum` and many more. Use `quote` for strings that might look like numbers or booleans.

Conditions and loops

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ .Release.Name }}
spec:
  rules:
    {{- range .Values.ingress.hosts }}
    - host: {{ . | quote }}
    {{- end }}
{{- end }}
```

`if` renders a block only when the value is set (an empty string, zero, false or empty list count as false). `range` repeats a block for each item, with `.` bound to the current item. `with` changes `.` to a value if it is not empty. Inside `range` or `with`, use `$` to reach the root, for example `$.Release.Name`.

MARGA by EknathaLabs marga.eknathalabs.com

Helpers and toYaml

```
# templates/_helpers.tpl
{{- define "web.labels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
{{- end }}

metadata:
  labels:
    {{- include "web.labels" . | nindent 4 }}
spec:
  template:
    spec:
      containers:
        - name: web
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

`include` renders a named template and returns a string that you can pipe, unlike `template`. `toYaml` turns a value into YAML text and `nindent N` adds a newline and indents each line by N spaces; the indent must match the position in the file, or the YAML breaks.

Whitespace and failing early

A dash inside braces trims whitespace: `{{-` removes the space before, `-}}` after. Use it so that conditionals do not leave blank lines or break indentation. To stop users from installing without a mandatory value, use `required`:

```
image: "{{ required "image.repository is required" .Values.image.repository }}"
```

For friendlier errors on bad combinations, call `fail`. Always test with `helm template` and with several values files, including the case where optional blocks are disabled. Debug with `helm template --debug`, which prints the rendered text even when it is not valid YAML, and use `kubectl apply --dry-run=server` to check against the real API.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Templates: the Go template language in Helm fits together

Part	What it does	What goes wrong
Values and objects	The top-level objects are .Values (your values), .Release (name, namespace, revision, whether it is an upgrade), .Chart (Chart.yaml fields), and .Capabilities (cluster...	Forgetting quote on strings that look like numbers.
Conditions and loops	if renders a block only when the value is set (an empty string, zero, false or empty list count as false).	Rendering only with default values in tests.
Helpers and toYaml	include renders a named template and returns a string that you can pipe, unlike template.	Wrong nindent values.
Whitespace and failing early	A dash inside braces trims whitespace: {{- removes the space before, -}} after.	Use it so that conditionals do not leave blank lines or break indentation.

Flow: Values and objects to Conditions and loops / Conditions and loops to Helpers and toYaml / Helpers and toYaml to Whitespace and failing early

Incident: The empty ingress that blocked a rollout

Incident: *A chart upgrade failed in staging only, with a message about an unknown field.*

```
$ helm upgrade shop-web ./shop-web -n shop -f values-staging.yaml
Error: UPGRADE FAILED: error validating "": error validating data: unknown field "rules" in io.k8s \
    .api.networking.v1.IngressSpec
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ helm template ./shop-web -f values-staging.yaml | sed -n 40,50p
spec:
  rules:
```

With ingress.hosts empty, the range produced no items, but the template still rendered an empty rules: key with bad indentation. The fix wrapped the whole Ingress in a condition that checks both enabled and a non-empty host list, and the team added a CI job that renders the chart with several values files (all optional features on, all off) and validates the output.

Common mistakes

- Wrong nindent values
- Forgetting quote on strings that look like numbers
- Rendering only with default values in tests
- Putting complex logic in templates

Check yourself

What does the pipe in {{ .Values.x | quote }} do?

Answer: Passes the value into the quote function

What is the difference between include and template?

Answer: include returns a string that can be piped, template cannot

What does {{- do?

Answer: Trims the whitespace before the action

How can a chart force a required value?

Answer: Use the required function with a message

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Upgrade, rollback and release history

12 minutes. Upgrade releases safely with wait and atomic, read history, roll back to a known good revision, and uninstall cleanly.

Installing is the easy part. The real value of Helm shows when the application changes: each upgrade becomes a numbered revision, and if it goes wrong you can return to a previous one with a single command. Using the right flags turns a failed upgrade into an automatic rollback instead of a half-broken release at 02:00.

What you will learn

- Upgrade with upgrade --install and the safety flags
- Read helm history and understand revisions
- Roll back and know what a rollback does and does not restore
- Uninstall and keep or remove persistent data

Upgrade

```
helm upgrade --install shop-web acme/web \
  --version 2.4.2 \
  --namespace shop --create-namespace \
  -f values-prod.yaml --set image.tag=1.8.3 \
  --atomic --wait --timeout 5m
```

`upgrade --install` installs if the release does not exist and upgrades if it does, so one command works for first and later deployments. `--wait` makes Helm wait until Deployments are ready and Jobs have completed, up to the timeout. `--atomic` implies wait and **rolls back automatically** if the upgrade fails or times out. Without them, `helm upgrade` returns as soon as the objects are accepted, and a broken rollout is reported as success. Pair them with readiness probes that mean something.

History

```
helm history shop-web -n shop
REVISION  UPDATED                   STATUS      CHART      APP VERSION  DESCRIPTION
1         Mon Jan  5 09:10:11 2026  superseded  web-2.4.0  1.8.0        Install complete
2         Mon Jan  5 14:22:40 2026  superseded  web-2.4.1  1.8.2        Upgrade complete
3         Tue Jan  6 10:05:02 2026  failed      web-2.4.2  1.8.3        Upgrade "shop-web" failed
4         Tue Jan  6 10:07:30 2026  deployed    web-2.4.1  1.8.2        Rollback to 2
```

Every change creates a revision, stored as a Secret. Helm keeps ten by default; set `--history-max` to control it. `helm get values --revision 2` and `helm get manifest --revision 2` show exactly what was applied then, which is invaluable for finding what changed.

Rollback

```
helm rollback shop-web 2 -n shop --wait
```

A rollback creates a **new** revision (4 in the example) that re-applies the manifest of revision 2. It restores the Kubernetes objects Helm manages. It does **not** restore data: a database migration that already ran, a persistent volume that was changed, or a secret edited by hand. Plan upgrades so that the previous version can run against the current data (backward-compatible migrations), and use hooks or a separate step for migrations. Also remember that a rollback can fail if a release is stuck in `pending-upgrade`; the debugging lesson covers that.

Uninstall

```
helm uninstall shop-web -n shop --keep-history
```

Uninstall deletes the objects of the release. PersistentVolumeClaims created by StatefulSets are usually **kept** by Kubernetes, and CRDs installed from a chart's `crds/` directory are not removed by Helm, on

MARGA by EknathaLabs marga.eknathalabs.com

purpose. Check what remains with `kubectl get pvc,crd`. Use `--keep-history` if you may want to roll back or inspect the past. In production pipelines, protect critical releases from accidental removal with access controls on who can run uninstall.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Upgrade, rollback and release history fits together

Part	What it does	What goes wrong
Upgrade	upgrade --install installs if the release does not exist and upgrades if it does, so one command works for first and later deployments.	Upgrading without --wait or --atomic.
History	Every change creates a revision, stored as a Secret.	Assuming rollback restores data.
Rollback	A rollback creates a new revision (4 in the example) that re-applies the manifest of revision 2.	Also remember that a rollback can fail if a release is stuck in pending-upgrade; the debugging lesson covers that.
Uninstall	Uninstall deletes the objects of the release.	Uninstalling production releases without protection.

Flow: Upgrade to History / History to Rollback / Rollback to Uninstall

Incident: The upgrade that said Happy Helming

Incident: *A release reported success while all pods were crash-looping.*

```
$ helm upgrade shop-web acme/web -n shop -f values-prod.yaml
Release "shop-web" has been upgraded. Happy Helming!
$ kubectl get pods -n shop
shop-web-6c9d7b8f4-hq2lm   0/1   CrashLoopBackOff   5   4m
```

Without --wait, Helm returned as soon as the manifests were accepted. The pipeline marked the deployment green, and nobody checked the pods. The team added --atomic --wait --timeout 5m to every upgrade, a post-deploy smoke test, and an alert on pods not ready, so a bad release now rolls back by itself within minutes.

Common mistakes

- Upgrading without --wait or --atomic
- Assuming rollback restores data
- Letting history grow or vanish unexpectedly
- Uninstalling production releases without protection

Check yourself

What does helm upgrade --install do?

Answer: Installs the release if missing, otherwise upgrades it

What does --atomic do on failure?

Answer: Rolls the release back automatically

What does helm rollback create?

Answer: A new revision that re-applies an older manifest

Which command shows what changed in an older revision?

Answer: helm get manifest RELEASE --revision N

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Values per environment and keeping secrets out

13 minutes. Organise values files for dev, staging and production, validate them, and handle secrets without committing them to version control.

Once a chart works, the next question is how to configure it for several environments without copy-paste and without leaking a database password into a repository. The answer is a layered set of values files, validation, and a clear rule: values carry configuration, and secrets arrive from somewhere else.

What you will learn

- Layer values files: base, environment and one-off
- Validate values with a schema and render in CI
- Why secrets do not belong in values files
- Safe patterns: existing secrets, external secret operators and encrypted files

Layering

A repository layout that scales:

```
deploy/shop-web/
  values.yaml           shared settings for all environments
  values-dev.yaml       smaller replicas, debug logging
  values-staging.yaml
  values-prod.yaml      replicas, resources, hostnames, pod disruption budget

helm upgrade --install shop-web acme/web -n shop \
  -f deploy/shop-web/values.yaml -f deploy/shop-web/values-prod.yaml
```

Later files override earlier ones. Keep the environment files short: only what differs. Review changes to `values-prod.yaml` with the same care as application code, because changing a replica count or a resource limit is a production change.

Validate

Add `values.schema.json` to the chart so wrong types or missing keys fail at install time:

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["image"],
  "properties": {
    "replicaCount": { "type": "integer", "minimum": 1 },
    "image": {
      "type": "object",
      "required": ["repository"],
      "properties": { "repository": { "type": "string" }, "tag": { "type": "string" } }
    }
  }
}
```

In CI, render every environment (`helm template -f ... | kubeconform` or a policy tool) so a mistake is found in the pull request, not during the deploy.

Secrets and values

Values are stored in release history (as Secrets, base64 encoded, not encrypted by Helm), appear in `helm get values`, and often sit in a repository. A password in `values-prod.yaml` therefore ends up in git history, in CI logs and in anyone's terminal who has read access to the release. Do not put secrets in values files or in `--set` arguments that appear in logs.

Better patterns

MARGA by EknathaLabs marga.eknathalabs.com

- **Reference an existing Secret**: the chart takes `existingSecret: shop-db` and the Secret is created separately, by an operator or a secure pipeline step.
- **External secrets**: a controller reads from a secret manager and creates the Kubernetes Secret; the chart only names it.
- **Encrypted files**: tools such as SOPS keep encrypted values in the repository and decrypt them in the pipeline with a key you control.
- **Sealed secrets**: encrypted objects that only the cluster can decrypt.

Whatever you choose, rotate any secret that was ever committed, restrict who can read Secrets in the namespace, and consider enabling encryption at rest for the cluster's Secret storage. In the chart, mount the secret as an environment variable or file by reference, never by value.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Values per environment and keeping secrets out fits together

Part	What it does	What goes wrong
Layering	A repository layout that scales.	Copy-pasted environment files with unnoticed drift.
Validate	Add values.schema.json to the chart so wrong types or missing keys fail at install time.	Secrets in values files or --set.
Secrets and values	Values are stored in release history (as Secrets, base64 encoded, not encrypted by Helm), appear in helm get values, and often sit in a repository.	Giving everyone read access to release Secrets.
Better patterns	Reference an existing Secret: the chart takes existingSecret: shop-db and the Secret is created separately, by an operator or a secure pipeline step.	No validation of values.

Flow: Layering to Validate / Validate to Secrets and values / Secrets and values to Better patterns

Incident: The password in helm get values

Incident: *A contractor with read access to one namespace could see the production database password.*

```
$ helm get values shop-web -n shop
USER-SUPPLIED VALUES:
db:
  host: pg.internal
  password: Sup3rS3cret!
```

The chart took the password as a value, so it was stored in the release Secret and printed by helm get values. The password was rotated, the chart was changed to reference an existing Secret, and an external secrets controller now creates that Secret from the secret manager. The team also tightened RBAC so that reading release Secrets requires an explicit role.

Common mistakes

- Secrets in values files or --set
- Copy-pasted environment files with unnoticed drift
- No validation of values
- Giving everyone read access to release Secrets

Check yourself

Which values file wins when two are given with -f?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: The later one

Where does Helm 3 keep the values of a release?

Answer: In the release Secret in the cluster

A safer way to give a chart a password?

Answer: Reference an existing Kubernetes Secret created elsewhere

What does values.schema.json provide?

Answer: Validation of values at install and lint time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Dependencies, subcharts and registries

13 minutes. Compose charts from dependencies, package and publish charts to an OCI registry, and version them correctly.

Your application needs a database or a cache, and someone has already published a chart for it. Helm lets a chart declare dependencies, so one install can bring up the whole stack. It also lets you publish your own charts to the same registry where you keep container images. Dependencies save time and add risk, so it matters how you pin, review and update them.

What you will learn

- Declare dependencies and update them
- Pass values to subcharts and use conditions
- Package and push a chart to an OCI registry
- Version, pin and update dependencies safely

Declare dependencies

```
# Chart.yaml
dependencies:
  - name: redis
    version: "19.6.2"
    repository: https://charts.example.com
    condition: redis.enabled

helm dependency update shop-web      # downloads into charts/ and writes Chart.lock
helm dependency build shop-web       # rebuild from Chart.lock (use in CI)
```

`Chart.lock` records exact versions and digests, like a lockfile. Commit it. In CI use `helm dependency build` so the build uses exactly what was reviewed. The dependency becomes a **subchart** installed together with your chart in the same release.

Passing values

Values for a subchart go under its name:

```
redis:
  enabled: true
  auth:
    existingSecret: shop-redis
  architecture: standalone
```

The `condition` in `Chart.yaml` lets users turn the subchart off, for example when production uses a managed cache. A subchart cannot read the parent's values, but a parent can pass them down, and `global:` values are visible to all subcharts. Remember that a subchart's own defaults may not be production safe (persistence off, no replicas, no auth); read them like any chart.

OCI registries

Helm can store charts in the same kind of registry as container images:

```
helm registry login registry.example.com
helm package shop-web                      # shop-web-0.3.1.tgz
helm push shop-web-0.3.1.tgz oci://registry.example.com/charts
helm install shop-web oci://registry.example.com/charts/shop-web --version 0.3.1 -n shop
helm pull oci://registry.example.com/charts/shop-web --version 0.3.1
```

OCI gives you the registry's access control, immutability settings and signing tools, and one place for images and charts. Classic HTTP repositories with an `index.yaml` still work and are served by tools such as ChartMuseum or static hosting. Choose one and document it.

MARGA by EknathaLabs
marga.eknathalabs.com

Versions and updates

Chart `version` is SemVer: raise MAJOR for breaking value changes, MINOR for new features, PATCH for fixes. Pin dependency versions exactly, or with a narrow range, and update on purpose: open a pull request that bumps the version, shows `helm template` differences, and runs your tests. Automated tools can propose these updates. Subscribe to release notes of important charts, watch for deprecations of Kubernetes APIs, and keep an internal mirror of external charts you depend on, so a deleted upstream does not stop deployments.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Dependencies, subcharts and registries fits together

Part	What it does	What goes wrong
Declare dependencies	Chart.lock records exact versions and digests, like a lockfile.	Not committing Chart.lock.
Passing values	Values for a subchart go under its name.	Trusting subchart defaults for production.
OCI registries	Helm can store charts in the same kind of registry as container images.	Depending on third-party repositories without a mirror.
Versions and updates	Chart version is SemVer: raise MAJOR for breaking value changes, MINOR for new features, PATCH for fixes.	Floating dependency ranges.

Flow: Declare dependencies to Passing values / Passing values to OCI registries / OCI registries to Versions and updates

Incident: The dependency that disappeared

Incident: *A deployment pipeline failed on a Monday morning with a missing chart error.*

```
$ helm dependency update shop-web
Error: can't get a valid version for repositories redis. Try changing the version constraint in Ch \
    art.yaml
$ curl -sI https://charts.old-example.com/index.yaml | head -1
HTTP/2 404
```

The upstream repository had been moved and the old URL returned 404, so every build failed. The team mirrored all external charts into their own registry, switched dependencies to the mirror, and used Chart.lock with digests. A weekly job checks the upstream for new versions and opens a pull request.

Common mistakes

- Floating dependency ranges
- Not committing Chart.lock
- Trusting subchart defaults for production
- Depending on third-party repositories without a mirror

Check yourself

What is a subchart?

Answer: A chart installed as a dependency within the parent's release

What does Chart.lock record?

Answer: The exact resolved dependency versions

How do you publish a chart to an OCI registry?

Answer: helm package, then helm push to an oci:// URL

How do you let users disable a subchart?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Add a condition such as redis.enabled in Chart.yaml dependencies

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Debugging Helm releases, hooks and tests

14 minutes. Diagnose failed installs and upgrades: render errors, stuck releases, hook problems and drift, and verify releases with helm test.

Helm failures come in a few families: the template does not render, the manifests render but Kubernetes rejects them, the manifests apply but the pods never become ready, or the release itself is stuck in a bad state. Each has a distinct signature and a distinct tool. A short checklist saves a lot of time and avoids the worst reflex, which is deleting things until it works.

What you will learn

- Separate render errors from API errors and rollout errors
- Fix a release stuck in pending-upgrade
- Understand hooks and use helm test
- Detect drift and compare with helm diff

Where did it fail?

Work from the left of this pipeline: render, validate, apply, roll out.

```
helm lint ./chart -f values.yaml
helm template rel ./chart -f values.yaml --debug | less     # render problems
helm template rel ./chart -f values.yaml | kubectl apply --dry-run=server -f -   # API problems
helm upgrade rel ./chart -f values.yaml --wait --timeout 5m --debug
kubectl get events -n NS --sort-by=.lastTimestamp | tail
kubectl rollout status deploy/NAME -n NS
kubectl describe pod POD -n NS
```

A message like `YAML parse error` or `nil pointer evaluating interface` is a **render** problem (look at the template and values). `unknown field` or `invalid value` is an **API** problem (the manifest is wrong for the cluster version). `timed out waiting for the condition` is a **rollout** problem: the objects were accepted but pods are not ready, so debug them with kubectl as in the Kubernetes debugging lesson.

Stuck releases

If an upgrade was interrupted (a CI job was killed), the release can stay in `pending-upgrade` or `pending-install`, and the next command says `another operation (install/upgrade/rollback) is in progress`.

```
helm history rel -n NS            # find the stuck revision and the last deployed one
helm rollback rel 7 -n NS         # roll back to the last good revision (often fixes it)
# last resort: remove the stuck revision record, never the workload
kubectl delete secret sh.helm.release.v1.rel.v9 -n NS
```

Deleting the release Secret of the stuck revision removes only Helm's record of it. Do this only after checking the history, and prefer a rollback. Also look for **ownership errors**, such as `invalid ownership metadata`, which mean an object already exists and was created outside this release: adopt it with the Helm labels and annotations or remove the duplicate.

Hooks and tests

Hooks run Jobs or Pods at points in the lifecycle, marked by annotations:

```
metadata:
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "0"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
```

Common uses are database migrations (`pre-upgrade`) and cleanup. Hooks are not part of the release's tracked objects, so they are not removed on uninstall unless the delete policy says so. A hook that fails fails the whole upgrade: check its Job logs. Chart tests are Pods with `helm.sh/hook: test`; run them with

`helm test rel -n NS` as a smoke test after each deploy.

Drift and diff

Someone edited a Deployment with kubectl, and the next upgrade surprises everyone. Compare before you apply:

```
helm plugin install PLUGIN_URL   # install once; see the plugin project page
helm diff upgrade rel ./chart -f values.yaml -n NS
helm get manifest rel -n NS | kubectl diff -f -
```

The diff plugin shows what an upgrade would change in the cluster compared with the stored release. Add it to the pipeline, and consider an operator or controller that continuously applies the chart from version control so the cluster state is always reconciled to what was reviewed. Keep `kubectl edit` for investigations, not for production changes.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Debugging Helm releases, hooks and tests fits together

Part	What it does	What goes wrong
Where did it fail?	Work from the left of this pipeline: render, validate, apply, roll out.	Killing pipeline jobs during a helm operation.
Stuck releases	If an upgrade was interrupted (a CI job was killed), the release can stay in pending-upgrade or pending-install, and the next command says another operation...	Deleting resources by hand to make an upgrade pass.
Hooks and tests	Hooks run Jobs or Pods at points in the lifecycle, marked by annotations.	Ignoring failed hooks.
Drift and diff	Someone edited a Deployment with kubectl, and the next upgrade surprises everyone.	Making manual edits in the cluster.

Flow: Where did it fail? to Stuck releases / Stuck releases to Hooks and tests / Hooks and tests to Drift and diff

Incident: Another operation is in progress

Incident: *A pipeline failed on every run after a runner was terminated in the middle of a deploy.*

```
$ helm upgrade --install shop-web acme/web -n shop -f values-prod.yaml
Error: UPGRADE FAILED: another operation (install/upgrade/rollback) is in progress
$ helm history shop-web -n shop | tail -3
8   Mon Jan 5 10:10:01 2026  deployed         web-2.4.1  1.8.2  Upgrade complete
9   Mon Jan 5 10:21:33 2026  pending-upgrade  web-2.4.2  1.8.3  Preparing upgrade
```

Revision 9 had been left pending when the runner died. A rollback to revision 8 reset the state, and the next upgrade succeeded. The team then set a pipeline timeout that is longer than helm's own timeout, so the job is not killed mid-flight, added --atomic, and wrote this procedure in the runbook so on-call does not delete anything it does not understand.

Common mistakes

- Deleting resources by hand to make an upgrade pass
- Killing pipeline jobs during a helm operation
- Ignoring failed hooks
- Making manual edits in the cluster

Check yourself

What kind of failure is "YAML parse error" from helm template?

Answer: A render problem in the template or values

MARGA by EknathaLabs marga.eknathalabs.com

What is the first safe fix for pending-upgrade?

Answer: helm rollback to the last deployed revision

How do you run a chart smoke test?

Answer: helm test RELEASE

How do you see what an upgrade would change?

Answer: helm diff upgrade (plugin) or compare the manifest with kubectl diff

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Testing and linting Helm charts

12 minutes. Catch chart errors before release with helm lint, template rendering, schema validation, unit tests and tests on a real cluster.

Charts are templates, and templates fail in ways you only see after rendering. A short set of checks, run on every pull request, finds most problems before a release.

What you will learn

- Linting and rendering charts in CI
- Validating output against Kubernetes schemas
- Constraining values with values.schema.json
- Install and upgrade tests on a temporary cluster

Lint and render

```
helm lint ./web -f ./web/ci/values-prod.yaml
helm template shop ./web -f ./web/ci/values-prod.yaml > /tmp/rendered.yaml
kubeconform -strict -summary /tmp/rendered.yaml
```

`helm lint` checks structure and common mistakes. `helm template` renders without a cluster, and a schema checker such as kubeconform validates the output against Kubernetes API schemas. Render with every values file you ship, since each can take different template paths.

Schema for values

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["image"],
  "properties": {
    "replicaCount": {"type": "integer", "minimum": 1},
    "image": {
      "type": "object",
      "required": ["repository", "tag"],
      "properties": {
        "repository": {"type": "string"},
        "tag": {"type": "string"}
      }
    }
  }
}
```

A `values.schema.json` file makes Helm reject bad values early with a clear message instead of failing during rendering or deployment.

Unit and cluster tests

Template unit tests, for example with the helm-unittest plugin, assert on the rendered output. Chart-testing runs `helm install` and `helm upgrade` against a temporary kind cluster, then runs `helm test`.

```
ct lint --config ct.yaml
ct install --config ct.yaml
helm test shop -n shop
```

Test **upgrades** from the previous chart version too, because most real failures are upgrade failures. Fail the pull request when any step fails.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Testing and linting Helm charts fits together

Part	What it does	What goes wrong
Lint and render	helm lint checks structure and common mistakes.	Rendering with default values only.
Schema for values	A values.schema.json file makes Helm reject bad values early with a clear message instead of failing during rendering or deployment.	No schema for values.
Unit and cluster tests	Template unit tests, for example with the helm-unittest plugin, assert on the rendered output.	Skipping tests because the chart is small.

Flow: Lint and render to Schema for values / Schema for values to Unit and cluster tests

Incident: The label that disappeared

Incident: *A refactor removed a selector label and the upgrade failed in production.*

```
Error: UPGRADE FAILED: cannot patch "web" with kind Deployment
spec.selector: Invalid value: field is immutable
rolled back to revision 7
```

The Deployment selector labels are immutable, and a template change altered them. Linting did not notice. The team added an upgrade test from the last released chart version, which catches this class of error before release.

Common mistakes

- Testing only fresh installs
- Rendering with default values only
- No schema for values
- Skipping tests because the chart is small

Check yourself

What does helm template do?

Answer: Renders manifests without a cluster

What does values.schema.json do?

Answer: Rejects invalid values early

Which test finds immutable field errors?

Answer: An upgrade test

Why render with every values file?

Answer: Different values take different paths

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 10. Library charts and reuse across services

12 minutes. Avoid copying templates between charts by using named templates, library charts and umbrella charts, and know the trade-offs.

Most services deploy with the same shape: a Deployment, a Service and some config. Helm gives you ways to share that shape so teams customize only what is different.

What you will learn

- Named templates and include
- Library charts and how service charts use them
- Umbrella charts and when they fit
- Versioning shared charts without breaking users

Library chart

```
# common/Chart.yaml
apiVersion: v2
name: common
type: library
version: 1.6.0

# common/templates/_deployment.tpl
{{- define "common.deployment" -}}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount | default 2 }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      securityContext:
        runAsNonRoot: true
      containers:
        - name: app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
{{- end -}}

# service chart: templates/deployment.yaml
{{ include "common.deployment" . }}

# service chart: Chart.yaml
dependencies:
  - name: common
    version: 1.x.x
    repository: oci://registry.example.com/charts
```

A library chart cannot be installed by itself. It only exports named templates. The service chart keeps its own values and a short template that calls the shared one.

Umbrella charts

An **umbrella chart** lists other charts as dependencies and installs them together, which suits a set of services that always deploy as one unit. It is a poor fit for independent services because one release then covers everything.

MARGA by EknathaLabs
marga.eknathalabs.com

Versioning

Use semantic versions for the library. Add new options with safe defaults, and treat changes in rendered output as breaking. Test the library against sample charts in CI, and let teams upgrade at their own pace.

```
helm dependency update ./orders
helm template orders ./orders | head -30
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Library charts and reuse across services fits together

Part	What it does	What goes wrong
Library chart	A library chart cannot be installed by itself.	Huge library templates with many switches.
Umbrella charts	An umbrella chart lists other charts as dependencies and installs them together, which suits a set of services that always deploy as one unit.	Using umbrella charts for independent services.
Versioning	Use semantic versions for the library. Add new options with safe defaults, and treat changes in rendered output as breaking.	New library options without defaults.

Flow: Library chart to Umbrella charts / Umbrella charts to Versioning

Incident: One library change, forty broken releases

Incident: *A new required value in the shared chart broke every service.*

```
Error: template: orders/templates/deployment.yaml:12
nil pointer evaluating interface {}.tolerations
40 pipelines failed
```

The library started reading a value that most services did not define and had no default. Services using a version range picked it up automatically. The team added defaults for every new option, pinned dependency versions and ran sample charts in the library's CI.

Common mistakes

- New library options without defaults
- Unpinned dependency ranges
- Using umbrella charts for independent services
- Huge library templates with many switches

Check yourself

What type is a library chart?

Answer: library

How do service charts use shared templates?

Answer: include

When fit an umbrella chart?

Answer: Services that deploy together

What should new library options have?

Answer: Safe defaults

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Publishing and signing Helm charts

12 minutes. Publish charts to an OCI registry, sign and verify them and control who may release or consume charts.

A chart is deployed into production, so the registry that serves it is part of your supply chain. A few steps make releases reproducible and verifiable.

What you will learn

- Packaging and pushing to an OCI registry
- Signing charts and verifying signatures
- Immutable versions and promotion
- Access control for publishing

Package and push

```
helm package ./web --version 2.5.0 --app-version 1.9.0
echo "$REGISTRY_TOKEN" | helm registry login registry.example.com -u ci --password-stdin
helm push web-2.5.0.tgz oci://registry.example.com/charts
helm show chart oci://registry.example.com/charts/web --version 2.5.0
```

Charts are stored as OCI artifacts next to container images. The version in `Chart.yaml` identifies a release, and it must change for every release.

Sign and verify

```
cosign sign --yes registry.example.com/charts/web:2.5.0
cosign verify registry.example.com/charts/web:2.5.0 \
  --certificate-identity-regexp 'https://ci.example.com/.*' \
  --certificate-oidc-issuer https://ci.example.com
```

Signing with an identity from CI ties the chart to a specific pipeline. Helm also supports provenance files with `helm package --sign`. Verification is useful only if consumers actually check it, so verify in the deployment pipeline before install.

Immutable versions and promotion

Configure the registry so tags cannot be overwritten. Publish to a development repository, test, then **promote** the same artifact to a production repository, instead of rebuilding. Record the digest in release notes.

Access

Allow only the CI identity to push to the release repository. Give consumers read-only access, scan charts for misconfiguration and keep an inventory of which charts and versions are used where.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Publishing and signing Helm charts fits together

Part	What it does	What goes wrong
Package and push	Charts are stored as OCI artifacts next to container images.	Mutable chart versions.
Sign and verify	Signing with an identity from CI ties the chart to a specific pipeline.	Pushing from laptops.
Immutable versions and promotion	Configure the registry so tags cannot be overwritten.	Rebuilding for production instead of promoting.
Access	Allow only the CI identity to push to the release repository.	Check the Common mistakes list below this diagram for what usually goes wrong here.

MARGA by EknathaLabs
marga.eknathalabs.com

Flow: Package and push to Sign and verify / Sign and verify to Immutable versions and promotion / Immutable versions and promotion to Access

Incident: The overwritten version

Incident: *A chart version was replaced with different content.*

```
helm pull web --version 2.4.1
digest sha256:aa10...   (earlier: sha256:73fe...)
production templates changed with no new release
```

Mutable tags let someone push a fix over an existing version. Deployments that were reinstalled picked up the changed content. The team turned on immutable tags, required signed releases from CI and verified signatures before every production install.

Common mistakes

- Mutable chart versions
- Pushing from laptops
- Signing but never verifying
- Rebuilding for production instead of promoting

Check yourself

Where are charts stored in modern Helm?

Answer: OCI registries

Why make tags immutable?

Answer: Content stays identical for a version

Who should push release charts?

Answer: The CI pipeline identity

What does verification need?

Answer: A check before install

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Helm security and supply chain

12 minutes. Review third-party charts, scan rendered manifests, avoid risky hooks and values and enforce policy so Helm cannot deploy unsafe workloads.

Helm makes it easy to install complex software with one command. That convenience hides how much power a chart has, from RBAC to hooks to default privileges. A few habits keep that power under control.

What you will learn

- Reviewing third-party charts before use
- Scanning rendered manifests and images
- Controlling what Helm may create
- Secrets and values hygiene

Review before you install

```
helm pull oci://registry.example.com/vendor/redis --version 19.2.1 --untar
helm template cache ./redis -f my-values.yaml > rendered.yaml
grep -nE "privileged|hostNetwork|hostPath|ClusterRole|ClusterRoleBinding" rendered.yaml
```

Read the rendered output, not only the values. Look for privileged containers, host access, cluster-wide RBAC, hooks that run jobs and images from unknown registries. Pin both the chart version and image tags or digests.

Scan

```
trivy config rendered.yaml
trivy image --severity HIGH,CRITICAL registry.example.com/vendor/redis:7.2.4
kubeconform -strict rendered.yaml
```

Scan manifests for misconfiguration and images for vulnerabilities in CI, and fail on agreed severities.

Limit the blast radius

Run Helm with credentials limited to the target namespace. Disallow cluster-scoped objects for most teams, and let a platform team install those. Enforce **Pod Security Standards** and policies such as Kyverno or OPA Gatekeeper at admission, so a chart cannot create a privileged pod even if you forget to check.

Secrets

Do not put secrets in values files committed to Git. Use an external secret operator or encrypted values. Remember that Helm stores release data, including values, in cluster Secrets, so restrict who can read them.

```
kubectl get secrets -n shop -l owner=helm
helm get values shop -n shop
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Helm security and supply chain fits together

Part	What it does	What goes wrong
Review before you install	Read the rendered output, not only the values.	Unpinned chart and image versions.
Scan	Scan manifests for misconfiguration and images for vulnerabilities in CI, and fail on agreed severities.	Scan Scan manifests for misconfiguration and images for vulnerabilities in CI, and fail on agreed severities.
Limit the blast radius	Run Helm with credentials limited to the target namespace.	Using cluster-admin for Helm installs.
Secrets	Do not put secrets in values files committed to Git.	Secrets in committed values files.

Flow: Review before you install to Scan / Scan to Limit the blast radius / Limit the blast radius to Secrets

Incident: The chart that wanted root

Incident: *An untrusted chart defaulted to privileged containers.*

```
securityContext: privileged: true
hostPath: /var/run/docker.sock
ClusterRoleBinding to cluster-admin
```

The team had installed the chart with defaults to try it quickly. A scan of the rendered output showed three dangerous settings. They overrode them in values, restricted the install credentials and added admission policies that block privileged pods.

Common mistakes

- Installing charts with default values unseen
- Unpinned chart and image versions
- Secrets in committed values files
- Using cluster-admin for Helm installs

Check yourself

What should you read before installing a chart?

Answer: The rendered manifests

What stops privileged pods even if a chart asks for them?

Answer: Admission policy

Where does Helm store release data?

Answer: Secrets in the cluster

How limit Helm credentials?

Answer: To the target namespace

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Helm and Kustomize

Charts, releases, values, upgrade and rollback commands, template functions and Kustomize overlays.

Repositories and search

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm repo list
helm search repo nginx --versions | head
helm show chart bitnami/nginx
helm show values bitnami/nginx > default-values.yaml
```

Install, upgrade, roll back

```
helm install web bitnami/nginx -n shop --create-namespace -f values-prod.yaml
helm upgrade --install web bitnami/nginx -n shop -f values-prod.yaml --atomic --timeout 5m
helm list -n shop
helm status web -n shop
helm history web -n shop
helm rollback web 2 -n shop
helm uninstall web -n shop
```

Keep overrides in values files, not `--set`. `--atomic` rolls back a failed upgrade.

Inspect and debug

```
helm template web bitnami/nginx -f values-prod.yaml
helm template web ./chart --debug
helm lint ./chart
helm get values web -n shop
helm get manifest web -n shop
helm diff upgrade web bitnami/nginx -n shop -f values-prod.yaml
```

`helm diff` needs the diff plugin. `helm template` never touches the cluster.

Chart layout

```
shop/
  Chart.yaml          name, version, appVersion
  values.yaml         defaults
  templates/
    deployment.yaml
    service.yaml
    _helpers.tpl      named templates
  charts/             dependencies

helm create shop
helm package ./shop
helm dependency update ./shop
helm push shop-0.1.0.tgz oci://registry.example.com/charts
```

Template syntax

```
{{ .Values.replicaCount }}                    read a value
{{ .Release.Name }}  {{ .Release.Namespace }} release info
{{ include "shop.fullname" . }}               call a helper
{{ .Values.image.tag | default "latest" | quote }}
{{- if .Values.ingress.enabled }} ... {{- end }}
{{- range .Values.hosts }} - {{ . }} {{- end }}
{{- toYaml .Values.resources | nindent 12 }}
```

`{{-` trims whitespace before the tag. Indentation errors are the most common bug.

Kustomize

```
kubectl kustomize overlays/production
kubectl apply -k overlays/production
kubectl diff -k overlays/production
kustomize build overlays/production | kubectl apply -f -

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  - ../../base
namespace: shop
images:
  - name: registry.example.com/shop/api
    newTag: 1.8.2
patches:
  - path: replica-patch.yaml
```

Base files stay valid YAML; overlays patch them per environment.

Find and inspect charts

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo nginx --versions | head
helm show values bitnami/nginx > values-default.yaml
helm show chart bitnami/nginx
helm pull bitnami/nginx --untar --version 18.2.0
helm lint ./mychart
helm template web ./mychart -f values-prod.yaml | less
```

> **Warning:** *A chart is code that creates cluster resources. Read the templates and the values of third-party charts before installing, and pin the chart version with* `--version`*.*

Install, upgrade, roll back

```
helm install web ./mychart -n shop --create-namespace -f values-prod.yaml
helm upgrade --install web ./mychart -n shop -f values-prod.yaml --atomic --timeout 5m
helm upgrade web ./mychart -n shop --reuse-values --set image.tag=1.5
helm diff upgrade web ./mychart -n shop -f values-prod.yaml
helm list -A
helm status web -n shop
helm history web -n shop
helm rollback web 3 -n shop --wait
helm get values web -n shop --all
helm get manifest web -n shop
helm test web -n shop
```

> **Warning:** `--reuse-values` *keeps old values and ignores new chart defaults, which can hide a broken upgrade. Prefer passing the full values file each time.*

> **Warning:** `helm upgrade --install` *without* `--atomic` *can leave a half-upgraded release. Use* `--atomic` *and* `--timeout`*, and check* `helm history` *after a failure.*

> **Warning:** `helm diff` *is a separate plugin (*`helm plugin list`*). Install the helm-diff plugin before touching production, so you see the change before you apply it.*

Remove

```
helm uninstall web -n shop --dry-run
helm uninstall web -n shop
helm uninstall web -n shop --keep-history
```

> **Danger:** `helm uninstall` *deletes every resource of the release, and persistent volume claims may follow depending on the chart. Back up data first. CRDs installed from a chart's* `crds/` *folder are not removed by Helm.*

MARGA by EknathaLabs
marga.eknathalabs.com